Hiero Workflow App : Architecture-First Response

Pre-interview task | Darshit | LFDT Mentorship 2026

Please go through the following Website: github-workflow-website.vercel.app

Thesis. Build a **GitHub App (Probot)** as the product centre with adapter-agnostic decision logic in `packages/core`. Start with `/assign` as the first product slice, then add PR quality, lifecycle, and review modules one vertical at a time. GitHub Actions serve as a compatibility / batch adapter over the same core.

Centralise: decision logic, schemas, config validation, parity tests, adapters, releases. **Keep local:** workflow YAML, triggers, permissions, checkout, harden-runner, policy. **Never rewrite:** SDK security controls or maintainer governance.

Q1. Well-Architected Principles Applied to Hiero [4 pts : K=2, R=2]

[K = 2: depth across six principles; R = 2: each principle applied specifically to Hiero's actual problem]

Clear responsibility. SDK repos own triggers, permissions, checkout, harden-runner, secrets, and local policy. The central GitHub App owns automation decisions, schemas, adapters, and releases. No step hides a security control. Hiero's inconsistency stems from this violation, retry logic and security assumptions are scattered across per-repo YAMLs instead of owned centrally.

Secure by design. Webhook HMAC-SHA256 verification at the listener boundary; installation-scoped tokens; CODEOWNERS on config; fail closed on missing or invalid config; strict command parsing.

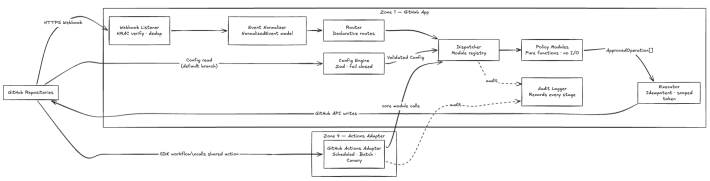
Low coupling / high cohesion. Decision logic lives in `packages/core`; the Probot GitHub App and the Actions adapter are thin delivery layers over the same functions : behaviour tested once.

Observable and testable. Every automation supports structured logs (Pino), idempotent writes via bot markers, golden fixtures, and parity tests. No module reaches production before its parity fixtures are green. Hiero currently lacks a shared parity baseline, the same automation behaves differently in Python and C++ with no test to catch the drift.

Evolvable. New automations register in the core module registry; config schema versions with 30-day backward-compatible windows mean new fields never break existing SDK installations.

Microkernel foundation. `packages/core` is the decision kernel; Probot App and Actions adapter are pluggable shells, policy changes never require shell rewrites (Richards, 2015).

Q2. Proposed Final System Architecture [4 pts : R=2, A=2]



[R = 2] [A = 2] *Eight-stage pipeline:* Listener → Normalizer → Router → Dispatcher → Config Engine → Policy Module → Executor → Audit Logger. Config Engine feeds Dispatcher. Audit Logger receives from every stage. **SDK repos** own workflow control, security, and per-repo config; **GitHub Actions adapter** calls the same `packages/core` modules for batch / scheduled operations.

Invariant: centralise reusable decision logic; do not centralise repository control. The App reads per-repo config : it never owns it.

C4 L2 container	Responsibility
SDK repos	Workflow YAML, triggers, permissions, harden-runner, checkout, concurrency, policy config.
sdk-automations (GitHub App)	Probot App (primary webhook path), Actions adapter (batch path), `packages/core` (shared modules: Assignment Policy, PR Quality Policy), schemas, fixtures, releases.
GH Actions	Compatibility / batch path: scheduled runs and SDK-owned workflows calling same core.

Q3. Architectural Boundaries and Why They Exist [4 pts : R=2, E=2]

[R = 2]

Repo-local : YAML, permissions, checkout, harden-runner, concurrency. Remain local because these are security controls. Moving them hides the security surface from accountable team, as discussed with Rob.

Shared-core : (a) Registry/dispatcher, (b) utilities, (c) policy modules. Central to prevent duplication. If per-repo scripts duplicate logic, one bug fix requires N PRs across N repos with no consistency guarantee, which is Hiero's current state.

Policy : Labels, team handles, limits. Protected local config prevents code forks. If C++ label names are hardcoded in `packages/core`, Python must adopt them or fork, both worse than a config field.

Hosted-app : Exists because interactive commands (`/assign`) require real-time webhook response. GitHub Actions cannot respond within seconds. Batch stays in Actions; event-driven in App. Collapsing this forces a polling model.

Three-bucket migration filter. Every code unit is classified before moving: **(1)** shared-core candidate: common decision logic, moves to `sdk-automations`; **(2)** repo-local boundary: workflow YAML and security controls, permanent; **(3)** repo-specific policy: expressed as protected config contract, never a hardcoded central module. Any unit that cannot be cleanly classified without resolving policy differences between Python and C++ stays local until the behaviour map is complete.

[E = 2] **Cost of crossing each boundary: Repo-local** → central: hides permissions from the accountable team; one compromised workflow YAML can escalate across all installed repos. **Core** → per-repo: creates hidden code forks; policy drift defeats centralisation. **Policy** → hardcoded: forces C++ to adopt Python defaults (or vice versa), breaking SDK-specific governance. **App hosting skipped**: loses real-time webhook response; batch-only Actions cannot handle interactive commands like `/assign`.

Q4. Tooling Choices [6 pts : R=2, A=2, E=2]

[R = 2]

Area	Recommendation and justification
Caller runtime	**GitHub App (Probot) first.** Real-time processing for `/assign` and PR checks. Actions for batch. Raw Octokit requires building token refresh and webhook routing from scratch; Probot provides these primitives, reducing Phase 1 scope.
Hosting	**Containerised Node.js / TypeScript** (Railway) with Redis for dedup/cache. Redis `SET NX` atomic with TTL, no transaction required. **Not serverless:** GitHub App tokens carry a 1-hour TTL requiring a persistent in-process cache; cold-start latency breaches GitHub's <10s webhook SLA.
Core language	**Node.js / TypeScript + Octokit.** Type-safe; enforces module interfaces; bundles natively.
Shared logic	**`packages/core`.** Adapter-agnostic modules. Probot and Actions adapter call the same functions.
Config	**JSON / YAML + Zod schema.** Zod validates and generates TS types at runtime. JSON Schema needs separate type generation; Zod eliminates the schema-types sync problem.
Verification	**Unit + mock + fixtures + parity.** Each layer catches a different failure class: logic, API contracts, or real-world call paths.
Security ops	**Harden-Runner, CODEOWNERS, SHA pins, Dependabot, rollback docs.** Governance gate, not optional extras.

[A = 2] [E = 2] Every choice is justified by what it *prevents*. Tooling that cannot be pinned, audited, or rolled back is not acceptable in a security-sensitive automation path.

Q5. How This Solution Builds Iteratively [2 pts : A=2]

[A = 2]

1. **Architecture + App shell first.** Webhook listener, config engine, event routing, and fail-closed validation before any policy module is attached.
2. **`/assign` as the first product slice.** Forces the full App path: comment parsing, config load, policy evaluation, GitHub write, and audit. Bounded scope.
3. **`/assign` guards and GFI cap.** Skill prerequisites, open-assignment

limit, Good First Issue graduation cap : parity-tested against C++ and Python behaviour.

4. **PR quality module.** DCO, GPG, merge conflict, and issue-link checks with dashboard comments and status label management.
5. **Review-sync and lifecycle modules** follow once the App shell is trusted and `/assign` parity is proven.
6. **Retire local code only after three gates:** parity tests green, pilot evidence matches expectations, and maintainer sign-off.

Q6. Malicious Pull Request Risks and Safeguards [10 pts : K=4, R=3, A=3]

[K = 4] Highest-risk design area. The architecture assumes a malicious contributor controls: PR title / body, branch name, changed files, artefacts from untrusted code, and workflow / config changes in the PR branch. The App **always loads config from the default branch** via the GitHub API : never from the PR head : so attackers cannot inject policy overrides through a malicious PR. **Fork PR token:** GitHub automatically restricts `GITHUB_TOKEN` to read-only for fork-sourced `pull_request` events; SDK workflows must never gate write operations on this token for fork PRs. `pull_request_target` is the highest-risk trigger in this context : it runs with base-repo write permissions against untrusted fork code. This architecture handles `pull_request_review` only; `pull_request_target` is explicitly excluded.

Risk	Safeguard
Forged / replayed webhook	HMAC-SHA256 signature verification at the listener; delivery-ID deduplication via Redis `SET NX` (24h TTL); on store unavailability, fail closed.
Config / policy tampering	CODEOWNERS on `.github/hiero-automation.*`; schema-validate every invocation; reject unknown fields; fail closed on missing config (no silent defaults).
Command injection	Strict regex parser (`/^\s*assign\s*$/i`); no argument pass-through; actor-type and bot guards before processing.
Token / secret exfiltration	Installation-scoped tokens; harden-runner in SDK workflows; restrict / audit egress; never checkout PR head in privileged jobs.
Action / dependency compromise	Pin all actions by full commit SHA; bundle central actions; Dependabot and `npm audit` continuously.
Artifact poisoning via `workflow_run`	Validate artifact identity: check run source, ID, path, and checksum before consumption. Never treat untrusted PR artifacts as an authority.
Over-broad permissions	GitHub App requests minimum scopes: Issues (write), Pull Requests (read), Contents (read). Installation-scoped tokens; no org-wide access.
Bot loops / broad writes	Actor / bot guards, idempotent bot markers, scoped label allowlists, concurrency keys, rollback by disabling the App installation or pinning back a SHA.

[R = 3] [A = 3] **Systemic safeguards:** validate actor type, event type, repo, PR number, label allowlists, and config schema *before* acting : unknown config fails closed. All writes are idempotent and scoped to known bot markers only. Config is loaded from the default branch via GitHub API (never from PR head); rate-limit handling with exponential back-off centralised in the Executor : no module retries independently, preventing duplicate write risk. For `workflow_run`: validate artifact identity, run source, and paths locally before consumption, PR #2278 used this exact vector via a 'Save Build Metrics' step.

Fail-closed default: missing or malformed config → error logged, no state mutations occur. **Partial-write recovery:** the Executor checks current GitHub state before any retry : a label already present is not re-applied; a comment with an existing bot marker is not re-posted. Per-entity concurrency locks (Redis `SET NX` per issue / PR) serialise concurrent webhooks.

Q7. Configuration Standardisation vs. User Input [6 pts : K=2, R=2, A=2]

[K = 2] [R = 2]

Standardise centrally	Allow per-repo (with guardrails)
Config schema version, automation names, command syntax, bot-comment markers, idempotency logic, error categories, fail-closed defaults, release compatibility.	Label names, team handles, docs links, assignment limits, GFI graduation cap, skill hierarchy, enabled automations, schedules, repo-specific message links.

When Python and C++ diverge on a value, that value is repo-specific policy : never hardcoded into `packages/core`. Core handles the decision *shape*; the per-repo config supplies the policy *parameter*. Concrete example: hiero-sdk-cpp sets assignmentLimits.maxOpenAssignments: 2 and assignmentLimits.maxGfiCompletions: 5 in .github/hiero-automation.json. The Python repo does not contain .github/hiero-automation.json on main, so those limits are not configured there; treat them as repo-specific policy when present. The two repos use different assignment handlers, so the shared runAssign claim should not be assumed without additional evidence.

[A = 2] **Decision rule:** if the value is the same in Python and C++ today and *must* be the same for correctness : standardise. If it differs today for legitimate governance reasons : config field. If it differs today due to historical accident with no intentional policy behind it : standardise with a migration note.

Q8. Community Leverage Plan [10 pts : K=3, R=3, A=4]

[K = 3] **Public issue backlog.** Architecture becomes a contribution surface covering behaviour mapping, fixture collection, docs, config examples, golden tests, security review, and SDK adoption guides : work that does not require maintainers to triage risky rewrites. Every community issue must carry: clear scope, definition of done, and a phase label : without these, contributors drift into production-adjacent work. Production behavior changes (upstream `/assign` writes, C++ assignment mapping, lifecycle module config) are `maintainer-only` and never opened.

[R = 3] **Skill-based labels:** `good-first-issue` (docs / fixtures) · `beginner` (tests / samples) · `intermediate` (adapters / config) · `advanced` (security pilots / C++ investigation).

Phase	Issue	Owner	Gate
Now	"Collect /assign golden cases : Python SDK"	Community	None
Now	"Define Zod schema for assignment section"	Community	None
Now	"Document full-SHA pinning policy"	Community	None
Phase 2	"Threat-model `pull_request_target` artifact flows"	Maintainer	App shell proven
Phase 3	"Per-SDK adoption checklist template"	Community	/assign pilot stable
Phase 4	"PR quality fixture collection : DCO/GPG cases"	Community	/assign parity green
Never	Upstream writes, C++ assignment, production lifecycle behavior	Maintainer-only	Architecture proven

Built-in contributor progression. The `/assign` system is itself the community onboarding mechanism: skill labels and prerequisite gates mean contributors naturally progress from `good-first-issue` to `beginner` to `intermediate` without maintainer intervention. The App doesn't just automate maintainer work : it actively grows the contributor pipeline that feeds future maintainers.

First safe batch (open now): `/assign` behaviour matrix, golden fixtures, SHA pinning doc, schema contract, SDK adoption checklist : filed at [sdk-automations/issues].

Q9. Architecture vs. Immediate Development [4 pts : R=2, E=2]

[R = 2]

Option	Risk	Assessment
Skip architecture	High	Replicates existing problem: independent evolution, inconsistent security, no parity baseline.
Full freeze	Medium	Delays reversible work unnecessarily; fixtures, schema, and dry-run adapters carry no risk.
2-week arch + reversible work	**Low**	**Recommended.** Architecture gates production writes; reversible work runs in parallel.

[E = 2] **Allocation: Wk 1** : App shell skeleton, boundary definitions, threat model, config schema. **Wk 2** : `/assign` behaviour maps, guard matrix, rollback plan, community backlog.

Safe to begin now: App shell, config engine, `/assign` module, parity fixtures, docs. **Must wait:** upstream production enablement, SDK security control changes.

Phase 0 already complete: Python fork canary (run proof), central repo CI (CI proof), and Probot adapter with 68 passing tests across three workspaces (core: 55, Probot: 10, Actions adapter: 3).